Governed Claude Code for GTM Engineering

Oct 12, 2026 · @Andy P

Executive summary

Every GTM user gets Claude Code, and every model completion and MCP tool call routes through one Databricks-backed AI gateway that enforces hard org-level cost limits. Cost governance is layered on top of that ceiling at two more tiers: a role-based team plugin and each user's own harness config. Each lower tier can only tighten the tier above it.

On that governed base we run GTM engineering workflows: resolving and enriching partial event lists, tracking high-intent signals, and scoring accounts against a backtested ICP model. Slack is one more client surface; it carries messages but never calls a model or tool directly.

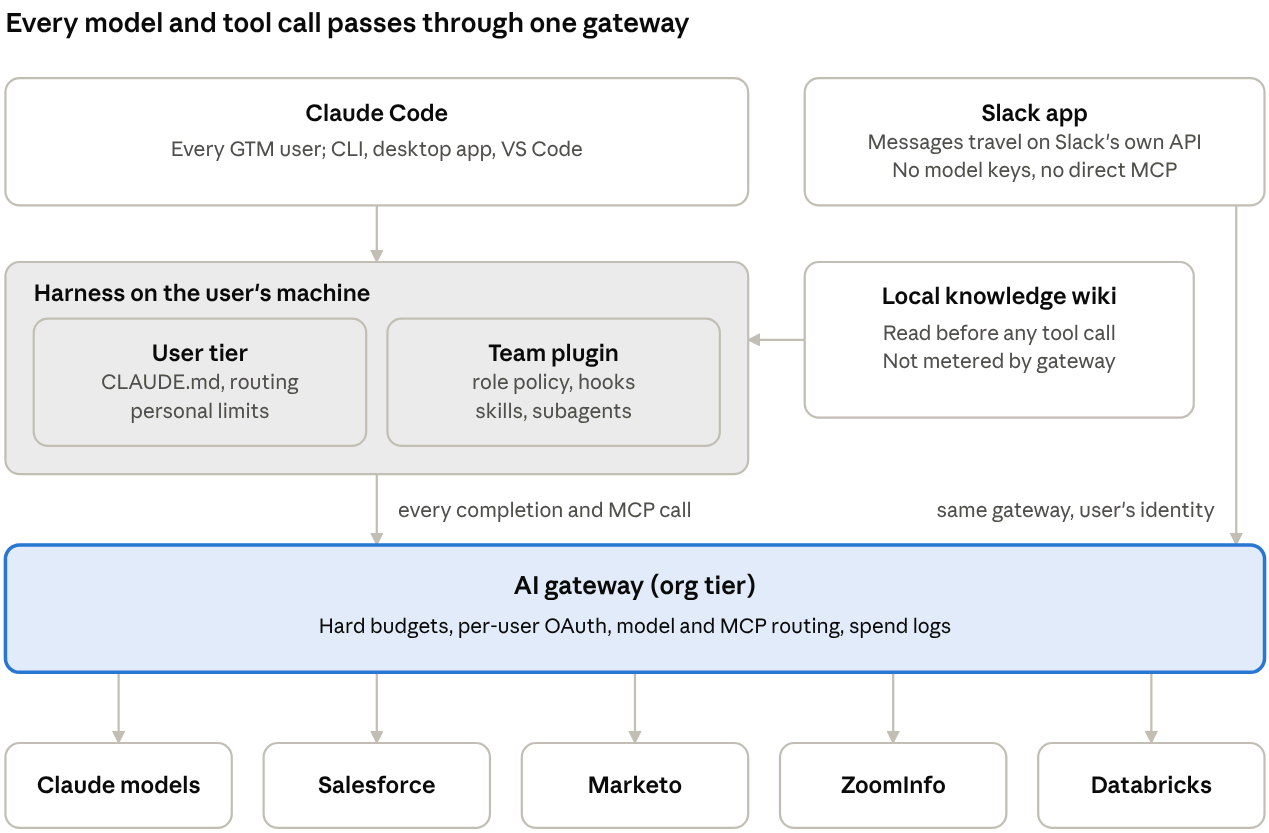
The bet is that Claude Code adds judgment where rules engines fail: is this the same company under another name, does this title fit our buyer, is this signal real intent. The MCP connectors are the hands, versioned skills hold the GTM logic, and the gateway keeps it safe to run org-wide.

Design principles

- One choke point: the gateway is the only path to models and MCP tools, so its caps cover total spend.
- Restrict downward, never loosen: org sets the ceiling, team and user can only narrow it.
- Logic in versioned skills, not vendor UIs: scoring and matching rules are inspectable, testable and diffable.
- Never fabricate contact data: an honest "unresolved" beats an invented email.
- Local context first, gateway last: preloaded knowledge cuts tool calls before they cost anything.

Architecture overview

Two surfaces, one choke point. Claude Code and the Slack app are clients; the harness shapes requests locally; the gateway is the only path to models and MCP servers.



architecture · 2 surfaces, 2 harness tiers, 1 gateway, 5 backends

The local wiki fills context before any call, so many tasks need few or no metered tool calls. Slack's messages never touch the gateway; only the model and tool work behind them does.

Three-tier cost governance

Cost is controlled at three tiers, each with a different owner, mechanism and failure mode. The gateway is the backstop that cannot be bypassed; the harness tiers shape behavior before a request ever reaches it.

Tier	Owner	Where it lives	Mechanism	What it controls	Bypassable by user?
Org	Platform / AI gateway owners	Databricks-backed AI gateway; Claude Code managed settings	Hard budgets, rate limits, OAuth identity, MCP routing	Total spend, which models and MCP servers exist at all	No

Tier	Owner	Where it lives	Mechanism	What it controls	Bypassable by user?
Team	GTM ops / team leads	A versioned Claude Code plugin per role	Hooks, skills, subagent definitions, permission rules	Which tools and models a role uses, per-role budgets	No, if delivered as managed config
User	Each person	User CLAUDE.md, user settings, personal hooks	Context, routing instructions, self-imposed limits	Waste reduction, personal budget, local-first retrieval	Yes, by design

Org tier (gateway). Every completion and every MCP call passes through it, so its caps cover total spend. Identity is per-user OAuth, and MCP tools are gated by each person's own access to the underlying system. Claude Code points at it via its base-URL setting, pinned in managed settings so users cannot reroute.

Team tier (plugin). Role logic is built once, bundled as a plugin, and rolled out per team without hand-editing anyone's config. It maps Claude implementations to the people who need them: an SDR role gets enrichment skills, a marketing ops role gets Marketo write access, and neither gets what it doesn't need.

User tier (harness). Lives entirely in the person's setup. Its main lever is cutting waste at the source: scoped CLAUDE.md context, local-first retrieval, and cheaper subagents for routine work.

One honest limit: harness hooks see tool calls, not token counts. Precise model spend is metered at the gateway; harness-level budgets are approximations (call counts, estimated cost per skill) that catch waste early and give users a readable reason.

Precedence and conflict resolution

The default rule is most-restrictive-wins, with the org gateway as a ceiling nobody can raise. Team and user tiers may tighten limits; neither may loosen them.

1. **Org ceiling is absolute.** A budget, model or MCP server the gateway disallows is unavailable regardless of any harness config.
2. **Deny beats allow at every tier.** A deny anywhere wins over an allow anywhere. This matches how Claude Code already evaluates permission rules.
3. **Budgets take the minimum.** Effective budget = min(org cap, team cap, user cap).

4. **Grants are explicit and bounded.** The one case that needs more than "most restrictive" is when a team must grant a capability a user has turned off, such as a required enrichment tool. Handle it with an explicit team allowlist delivered as managed config, valid only inside the org ceiling.
5. **Conflicts are logged, not silent.** When a lower tier is overridden, the hook says which tier decided and why, so users aren't debugging invisible rules.

Open decision: does the team tier only restrict, or may it also grant? Recommendation: restrict by default, with a small reviewed allowlist for role-essential tools.

Harness controls

Five harness mechanisms shape cost before a request reaches the gateway. Each has a distinct job.

Mechanism	Job	Cost lever	Typical tier
CLAUDE.md	Context and policy loaded every session	Scopes what a user is trying to do; carries local-first routing rules so fewer calls are needed	User (soul + project files); team ships a role section
AGENTS.md	Cross-tool convention for agent instructions	Same as CLAUDE.md for tools that read it; Claude Code auto-loads CLAUDE.md, so reference one from the other	Team / repo
Hooks	Code that runs at lifecycle events	Blocks, redirects or meters calls before they leave the harness	Team (plugin) and user
Subagents	Delegated workers with their own context	Push routine or narrow work to a smaller model; keep main context lean	Team (plugin)
YAML frontmatter	Per-skill and per-agent config	Pins model choice and allowed tools per agent or skill	Team (plugin)

Example subagent definition for routine enrichment on a smaller model, with tools limited to read-only lookups:

```
---
name: enrichment-matcher
description: Resolves partial event-list rows against ZoomInfo and
Salesforce. Use for identity matching, not for writes.
model: haiku
tools: mcp__zoominfo__search_contacts, mcp__salesforce__query
---
Match each row on name + company + title + region.
Return a confidence score 0-1 and the evidence for it.
Never write to Salesforce. Never invent an email or phone number.
```

Tool names above are illustrative; use the exact MCP tool names your connectors expose.

Context as a cost control. The knowledge-wiki design applies here: the session preloads a scoped subgraph (soul, project, the relevant account node) before any tool call. Filling context from local markdown is not a gateway call. One fatter prompt replaces many round-trips. The project CLAUDE.md can also carry a plan of investigation: the bounded set of calls a task may need, each tied to a skill with a known budget, so cost is predictable before inference.

Hooks for cost control

PreToolUse is the cost-control hook: it runs before a tool call executes and can deny it. The harness passes the call as JSON on stdin; the script returns allow or deny with a reason the user sees immediately, before anything reaches the gateway.

Lifecycle events used

- PreToolUse: budget and role checks; deny expensive or out-of-role MCP calls.
- PostToolUse: record the call to a local ledger for metering and reconciliation against gateway logs.
- SessionStart: preload scoped context and print the user's remaining budget.
- UserPromptSubmit: optional; warn when a request is likely to exceed budget.

Configuration (settings JSON; shipped in the team plugin's hooks file, or in managed settings to make it non-optional):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__.*",
        "hooks": [
          { "type": "command", "command": "python3
~/.claude/hooks/budget_gate.py" }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "mcp__.*",
        "hooks": [
          { "type": "command", "command": "python3 ~/.claude/hooks/ledger.py"
}
        ]
      }
    ]
  }
}
```

Skeleton budget gate (Python; SQLite ledger for a single machine, swap for Redis or a Databricks table for shared team budgets):

```
#!/usr/bin/env python3
import json, sys, os, sqlite3, datetime

event = json.load(sys.stdin)
tool = event.get("tool_name", "")
user = os.environ.get("USER", "unknown")

# Policy shipped by the team plugin: role -> allowed tools, daily call
budget, per-tool cost weights
policy = json.load(open(os.path.expanduser("~/.claude/gtm-policy.json")))
role = policy["users"].get(user, "default")
rules = policy["roles"][role]

def deny(reason):
    print(json.dumps({"hookSpecificOutput": {
        "hookEventName": "PreToolUse",
```

```
        "permissionDecision": "deny",
        "permissionDecisionReason": reason}}))
    sys.exit(0)

if not any(tool.startswith(p) for p in rules["allowed_tool_prefixes"]):
    deny(f"[team policy: {role}] {tool} is not enabled for your role.")

db = sqlite3.connect(os.path.expanduser("~/.claude/ledger.db"))
db.execute("CREATE TABLE IF NOT EXISTS calls (day TEXT, tool TEXT, weight
REAL)")
today = datetime.date.today().isoformat()
spent = db.execute("SELECT COALESCE(SUM(weight),0) FROM calls WHERE day=?",
(today,)).fetchone()[0]
weight = rules["tool_weights"].get(tool, 1.0)

budget = min(rules["daily_budget"], policy.get("user_overrides",
{}).get(user, float("inf")))
if spent + weight > budget:
    deny(f"[budget: {role}] Daily tool budget reached
({spent:.0f}/{budget:.0f}). Resets tomorrow.")

sys.exit(0)  # allow; ledger.py records the call in PostToolUse
```

Notes:

- `min()` implements the precedence rule: the user's own cap can only lower the team budget.
- Deny reasons name the tier that decided, so users know whether to ask their lead or change their own config.
- Weights are estimates per tool, set from gateway cost data. Recalibrate monthly against actual gateway spend.
- A hook that errors should fail closed for write tools and fail open for read tools; decide this explicitly.

Team-level role-based plugin

The team tier ships as one versioned plugin per GTM function, so role rules are built once and rolled out without touching anyone's personal config. It also carries the GTM skills, which makes governance and capability the same install.

Plugin layout

```
gtm-governance/
  .claude-plugin/plugin.json      # name, version, description
  hooks/hooks.json                # PreToolUse budget gate, PostToolUse
ledger, SessionStart preload
  scripts/budget_gate.py
  scripts/ledger.py
  policy/gtm-policy.json          # roles, allowed tool prefixes, budgets,
tool weights
  agents/enrichment-matcher.md    # small-model subagent, read-only tools
  agents/icp-scorer.md
  skills/bootstrap/SKILL.md       # sets up CLAUDE.md tiers idempotently
  skills/event-list-resolve/SKILL.md
  skills/intent-digest/SKILL.md
  skills/icp-score/SKILL.md
  .mcp.json                       # MCP servers, all pointed at the gateway
```

Role matrix (starting point)

Role	Enabled skills	MCP access	Default model	Daily budget
SDR / BDR	Account briefing, intent digest	Salesforce read, ZoomInfo read, Slack post	Small model for lookups	Low
AE	Account briefing, opportunity context	Salesforce read/write own records, ZoomInfo read	Standard	Medium
Marketing ops	Event-list resolve, intent digest, ICP score	Marketo read/write, Salesforce read/write, ZoomInfo read	Standard	High
GTM engineering	All	All, including admin tools	Standard; large on demand	Highest
Leadership	Briefings, reports	Read only	Standard	Medium

Budget levels are placeholders; set them from the first month of gateway cost data.

Mapping users to roles. The source of truth for role should be an existing system, not a hand-kept list: the Salesforce user profile or role, or the identity provider group. The policy file is generated from it, so access follows job changes automatically. Users who don't need Claude for GTM simply aren't assigned a role and get the default, which is read-only.

Slack as a governed surface

Slack is another client of the gateway, not a parallel brain. It has two planes, and only one of them is governed.

Plane	What travels on it	Goes through gateway?	Visible where
Message plane	DMs to the app, app replies, thread posts	No; Slack's own API	Slack UI
Model and tool plane	Model completions, MCP tool calls the agent makes	Yes, always	Gateway logs and budgets

Request flow

1. A user DMs or mentions the app. Slack delivers the event via Socket Mode or the Events API.
2. The app acknowledges within Slack's ~3-second window, before doing any work.
3. The app enqueues the job and calls the gateway with the user's identity, never a shared service account.
4. The gateway runs the completion and MCP calls under that user's budget and access.
5. The app posts the result into the thread asynchronously.

Identity mapping is the critical design point. Slack user ID maps to email, which maps to the gateway OAuth identity and the team role. If the Slack app used one service account, every Slack request would share one budget and one set of permissions, and both the cost tiers and the per-user data access would collapse.

The Slack app stays deliberately dumb: no model keys, no direct MCP connections, no business logic. Everything intelligent sits behind the gateway, where it is metered.

GTM engineering workflows

Three workflows come first, each running both event-triggered and on a schedule. All use the connected MCPs (Marketo local, Salesforce, ZoomInfo, Slack) through the gateway, with their logic in versioned skills.

1. Event-list resolve and enrich

The problem: live-event lists are loaded manually, often bypass Salesforce, and arrive with enough attributes to identify someone (name, company, title, region) but no email or phone. The pipeline either resolves a record to verified contact data or parks it honestly.

1. **Ingest.** A list lands (file drop, Slack upload, or a staging table). Event trigger: run immediately. Normalize columns and tag every row with the event's canonical campaign ID.
2. **Identity resolution.** Match each row against existing Salesforce leads and contacts first, then ZoomInfo. Claude's judgment handles fuzzy cases: company aliases, title variants, parent and subsidiary names. Output: a match, a confidence score and the evidence.
3. **Enrichment.** For matched rows, pull email and phone from ZoomInfo and firmographics for the account.
4. **Confidence gate.** Above threshold: write through Marketo first, so lifecycle and scoring stamp before the native sync to Salesforce, with campaign membership attached. Below threshold: send to a holding queue. Never write a fabricated or guessed email.
5. **Sweep.** A scheduled job retries the holding queue as ZoomInfo data refreshes. Rows still unresolved after a set age go to a human review list in Slack, with the attributes laid out.

Design choices:

- Marketo-first entry keeps the single-front-door dedup strategy intact; event lists stop being a back door.
- The holding queue should live in Databricks (or a Salesforce custom object if reps must see it). It must not be Salesforce leads with blank emails, which pollute the CRM and the audit.
- Threshold starts conservative (for example 0.85) and is tuned from human review outcomes.

2. High-intent signal tracking

A scheduled sweep plus event triggers collect signals from Marketo activity (form fills, high-value page visits, campaign responses) and ZoomInfo intent data. Each account is scored on recency, strength and breadth across its buying group.

- The definition of "high intent" lives in a versioned skill, so it is explicit, reviewable and tunable rather than buried in a scoring UI.
- Output: a ranked digest posted to the owning rep's Slack, and the account's next-best-action field updated.
- The trigger vocabulary is the attributable campaign-member status change (for example sent to responded), defined per campaign in the crosswalk.

3. ICP scoring model

The ICP is a weighted attribute model, not a static filter. The agent scores every account and lead against it, then backtests it.

- **Define:** firmographic, technographic and persona attributes, each with a weight.
- **Score:** every account and lead gets an ICP fit score, stored in Databricks and synced to a field.
- **Backtest:** compare scores against closed-won and closed-lost outcomes. Which attributes actually predicted revenue? Which ICP-fit accounts never converted?
- **Close the loop:** rep qualification outcomes on MQLs show whether the high-intent signals and the ICP are right, and the weights are revised from evidence.

The backtest is what makes this a model rather than a guess. It depends on populated attributes, which ties directly to deduplication: duplicates fragment attributes across copies.

Data foundations and dependencies

The workflows are only as good as the data layer under them, and that layer is what the attribution audit is building. Each workflow depends on specific audit deliverables.

Foundation	Where it lives	What depends on it
Canonical campaign ID and campaign crosswalk (SFDC campaign ID, Marketo program ID, lead source, attributable event, next-best-action)	Salesforce metadata table, mirrored in Databricks	Event-list tagging, intent triggers, attribution
Universal person key (identifier crosswalk folding every lead ID for a person)	Databricks	Identity resolution, dedup, ICP backtest
Governed lead source dimension	Salesforce + Databricks	Attribution, ICP loop (which sources produce qualified leads)
Enumerated action vocabulary (defined conversion actions)	Crosswalk	Intent definitions, next-best-action
Marketo-first single front door	Marketo forms and API intake	Dedup; lifecycle stamping before sync
Account node and ABM custom object	Databricks, synced to Marketo named accounts	Account-level intent, account NBA

System roles. Marketo handles capture, engagement and hand-raise origination. Salesforce is the operational system of record and carries the canonical campaign ID. Databricks is the analytical layer: crosswalks, attribution, ICP scoring, the holding queue.

Knowledge wiki. The local-first GTM wiki supplies the interpretive context the agent reasons through: business rules, ICP definitions, campaign intent. Slow-moving knowledge is preloaded; aggregates come from Databricks (Genie, potentially via its managed MCP endpoint and on Genie credits rather than the gateway meter); must-be-live single records come through the gateway last.

Write-back discipline. AI-generated narrative goes into task bodies and digests, not structured fields. Structured fields carry only computed values (scores, NBA codes, timestamps) so the measurement surface stays clean.

Observability, metrics and rollout

Spend is measured at the gateway, value is measured in GTM outcomes, and the two are joined per workflow so cost per result is visible.

Metrics

Metric	Source	Why it matters
Spend by user, role, skill and workflow	Gateway logs joined to hook ledger	Shows where money goes; calibrates hook weights
Calls denied by tier and reason	Hook output logs	Reveals budgets set too tight or roles missing a tool
Event-list resolution rate and time to resolve	Holding queue in Databricks	Core value of workflow 1
False-match rate from human review	Review outcomes	Tunes the confidence threshold
Intent digest to rep action rate	Salesforce tasks and activity	Whether signals are useful to reps
ICP backtest lift (win rate of high-fit vs low-fit accounts)	Databricks	Whether the ICP is right
Cost per resolved record, per qualified meeting	Gateway spend / outcomes	The ROI line for the executive summary

Rollout phases

1. **Pilot (GTM engineering and marketing ops).** Gateway caps live, hooks in log-only mode, event-list workflow on recent events. Gate: resolution rate and false-match rate measured on real lists.
2. **Enforce.** Hooks switch from log-only to deny. Role policy generated from the identity source. Gate: one month of spend data to set budgets and weights.
3. **Expand.** Plugin to SDR, AE and leadership roles; intent digest to Slack; ICP scoring with first backtest. Gate: digest action rate and backtest lift reviewed with sales leadership.
4. **Steady state.** Monthly weight recalibration, quarterly ICP revision, plugin versions released like code.

Open decisions and risks

Decisions to make

- [ ] Team tier: restrict only, or restrict plus a reviewed grant allowlist?
- [ ] Role source of truth: Salesforce profile/role, or identity provider groups?
- [ ] Shared budget store for team-level caps: Redis, Databricks table, or gateway-native?
- [ ] Holding queue location: Databricks only, or a Salesforce custom object reps can see?
- [ ] Starting confidence threshold for writing enriched contacts.
- [ ] Hook failure mode per tool class (fail closed for writes, open for reads?).
- [ ] Confirm Genie credits are billed separately from the gateway, and whether the managed Genie MCP endpoint is reachable.

Risks

Risk	Effect	Mitigation
Hooks approximate cost; gateway is the truth	Users hit gateway limits the harness didn't predict	Monthly recalibration of tool weights from gateway data
Hooks in user settings can be edited	User-tier controls are advisory	Put enforced hooks in managed settings; keep the gateway as the hard cap
Slack app built on a service account	Budgets and data access collapse to one identity	Per-user OAuth mapping from Slack ID to gateway identity
ZoomInfo match errors	Wrong contacts written into CRM	Confidence gate, holding queue, human review sampling
Event lists keep bypassing intake	Duplicates and lifecycle-blind records	Marketo-first intake plus a catch flow that stamps unstamped records
Sparse or duplicated attributes	ICP backtest is misleading	Sequence ICP work after dedup and the universal person key
Sync latency between SFDC and Marketo	Gating or intent on stale data	Timestamp every computed field; don't gate on stale opportunity data